

KidCode 2

WEEK 1: LEVEL UP: FUNCTIONS & LISTS

Day 1: Welcome Back & Skills Check-Up

Reviewing Year 1: variables, strings and methods, if statements, loops, lists, dictionaries, and functions. "Python Olympics" relay of short challenges. Setting up the workspace and a tidy KidCode project folder.

Day 2: Functions Power-Up

Default values, keyword arguments such as `greet(name="friend")`, and returning several values as a tuple. Functions that call other functions. Practice: "Mini Math Toolkit".

Day 3: Scope, Docstrings & Testing

Local vs. global variables, docstrings, and testing with `assert`: write the test first, then the function. Lab: "Bug-Proof Calculator", where every function passes its own tests.

Day 4: List Power Tools

Nested lists, `sorted()` and `.sort(key=...)`, `enumerate()`, `zip()`, and sets for unique values with `set()` and `.add()`. Practice: "Class Leaderboard" sorted by score.

Day 5: List Comprehensions

One-line list builders like `[x * 2 for x in numbers]`, filtering with `if`, and comprehensions over strings and words. Converting `for` loops into comprehensions. Practice: "Word Shouter" and "Even Filter".

WEEK 2: SECRET CODES & FILES

Day 6: Character Codes & Caesar Shifts

How computers store letters as numbers with `ord()` and `chr()`. Shifting letters with `%` to wrap around the alphabet and looping through strings. Practice: "Letter Shifter".

Day 7: Lab: Secret Agent Cipher Lab

Project 1. Building `encrypt(text, shift)` and `decrypt(text, shift)` with `assert` tests. Swapping coded messages, then a crack-the-code challenge that tries all 25 shifts.

Day 8: Reading Files

Opening files with `open()` and `with`, then `.read()`, `.readlines()`, and `for line in file`. File paths, the working folder, and `FileNotFoundError`. Practice: "Story Reader".

Day 9: Writing & Appending Files

File modes `"w"` and `"a"`, `.write()`, newline characters, and why write mode erases old data. Practice: "Diary Logger" adds an entry each run, and "High Score Keeper" remembers scores.

Day 10: Lab: Message Vault

Project 2. Combining the cipher with file reading and writing: save secret messages to a file and unlock them with a password. Swap vaults and try to crack them.

WEEK 3: ERRORS & DATA FORMATS

Day 11: try/except: When Things Go Wrong

Catching errors with `try` and `except`, and common exceptions (`ValueError`, `ZeroDivisionError`, `KeyError`, `FileNotFoundError`). Reading tracebacks. Practice: "Safe Divider".

Day 12: Bulletproof Input

Validation loops that re-ask until the input is good, `else` and `finally` blocks, and a reusable `ask_number()` function that no silly answer can break.

Day 13: CSV Files

Reading and writing spreadsheet-style `.csv` files with the `csv` module, rows as lists, and `DictReader` for rows as dictionaries. Practice: "Pet Registry".

Day 14: JSON Files

Saving and loading structured data with `json.dump()` and `json.load()`. How dictionaries and lists map onto JSON. Practice: "Game Settings" and a "Quiz Question Bank".

Day 15: Lab: Trivia Quiz 2.0

Project 3. Upgrading Year 1's quiz: questions loaded from a JSON file, high scores saved to disk, input protected with `try/except`, and one function per job. Pair review with a checklist.

WEEK 4: OBJECTS & CLASSES

Day 16: Everything Is an Object

Discovering that `str`, `list`, and `dict` are classes and that methods are functions belonging to objects. Exploring with `type()`, `dir()`, and `help()`. A class is a blueprint, like a cookie cutter.

Day 17: Your First Class

Writing a class with `class`, `__init__`, `self`, and attributes, then creating several instances. Practice: a Pet class with `name`, `species`, and `energy`.

Day 18: Methods & Readable Objects

Giving objects behaviors: methods like `feed()` and `play()` that change attributes, and methods that return values. Using `__str__` so `print(pet)` is readable.

Day 19: Objects Working Together

Storing objects in lists, objects that hold other objects (a Team with Players), and class vs. instance attributes. Practice: "Zoo Manager".

Day 20: Lab: Virtual Pet Care

Project 4. Designing a pet on paper, then coding hunger and happiness meters, a daily menu of actions, and saving the pet to a JSON file. Classmates trade pets and keep them alive.

WEEK 5: INHERITANCE & THE ARENA

Day 21: Inheritance: Parents & Children

Child classes that inherit from a parent class, `super().__init__()`, and overriding methods. Practice: an `Animal` parent with `Dog` and `Cat` children that each `speak()` differently.

Day 22: Polymorphism & `isinstance()`

Treating different objects the same way: looping through a mixed list of animals and calling the same method. Using `isinstance()` and planning class families on paper.

Days 23-24: Lab: Monster Battle Arena

Project 5. A `Monster` parent class with `name`, `hp`, and `attack`, plus `FireMonster` and `WaterMonster` children. A turn-based battle loop with random damage, type advantages, and a saved "Hall of Fame".

Day 25: Code Review & Bug Bounty

Swapping projects and hunting bugs with a checklist: clear names, docstrings, functions that do one job, and passing tests. Kind, specific feedback, then refactoring.

WEEK 6: LIBRARIES & MODULES

Day 26: Modules & the Standard Library

A module is a `.py` file of ready-made tools and a library is a collection of them. Using `import`, from `... import`, and `as`. Touring `math`, `random`, `datetime`, `string`, and `collections`.

Day 27: Your Own Module & the Docs

Moving helpers like the `cipher` and `ask_number()` into `kidtools.py` and importing them. Reading Python's official documentation: finding a function and reading its signature.

Day 28: Lab: Turtle Art Studio

Project 6. Using the built-in `turtle` library: `turtle` objects with methods such as `forward()`, `left()`, and `begin_fill()`. Loops and functions draw spirals, stars, and a `draw_flower()` function.

Day 29: pip & Third-Party Libraries

What `PyPI` is and how `pip install` works, and the difference between a module, package, and library. Installing `rich` for colorful text, reading a `README`, and fixing a failed install safely.

Day 30: Lab: Class Survey Dashboard

Installing `matplotlib` and drawing bar charts with `plt.bar()`, `titles`, `labels`, and `plt.show()`. Loading survey answers from a `CSV` file and charting the results.

WEEK 7: TEXT AS DATA

Day 31: Loading & Cleaning Text

Loading a public-domain book from Project Gutenberg as a text file. Cleaning it with `.lower()`, `string.punctuation`, and `.split()`, and finding unique words with `sets`.

Day 32: Counting Words with Counter

Using `collections.Counter` and `.most_common()` to rank words, and stop-word lists to skip "the" and "and". Practice: "Top 10 Words" for any text file.

Day 33: Lab: Word Frequency Chart

Charting a book's top words with `matplotlib` and comparing two books: which words define each story? Adding average word length and vocabulary size to the report.

Day 34: Comparing Texts with Sets

Set operations `&`, `|`, and `-` to find shared and unique words between two texts. Measuring lexical diversity (unique words divided by total words). Practice: "Author Fingerprint".

Day 35: Language Is Tricky: Bug Safari

Testing our code on tricky text: `.split(" ")` breaks on "Dr. Smith" and "U.S.A.", and `.split()` leaves punctuation stuck to words. Logging every failure as a cliffhanger for Year 3.

WEEK 8: CAPSTONE & SHOWCASE

Day 36: Capstone I: Book Detective Plan

Project 7. Planning a `TextAnalyzer` class that loads a book and reports total words, unique words, longest word, average word length, and top words. Sketching attributes and methods on paper.

Day 37: Capstone II: Building the Core

Coding the class: loading a file with `try/except`, cleaning text, counting with `Counter`, and one method per statistic. Testing each method with `assert`.

Day 38: Capstone III: Charts & Reports

Adding a `matplotlib` chart method, saving a full text report to a file, and letting users pick a book from a menu. Reusing functions from `kidtools.py`.

Day 39: Playtest, Polish & Debug

Swapping projects so testers can try to break them with odd files. Fixing bugs, adding docstrings, and rehearsing a three-minute demo.

Day 40: Showcase Day & Year 3 Preview

Presenting Book Detective reports to classmates and guests, plus a skills checklist. Preview of KidCode 3: the sentences that broke our code are what NLP and the `spaCy` library are built to solve.